



LECCIÓN 9: PLANIFICACIÓN DE PROYECTOS CALIDAD EN PROYECTOS

Ingeniería Informática



AGENDA

- ① Gestión de la Calidad
 - ② Aseguramiento de la Calidad en Proyectos Informáticos



GESTIÓN DE LA CALIDAD

- ◎ Calidad: Nivel en que un conjunto de características inherentes satisface los requisitos.
- ◎ Grado: Categoría asignada a un producto o servicio con un mismo uso funcional pero con características técnicas diferentes



UN FERRARI NUEVO



UN LAND ROVER USADO



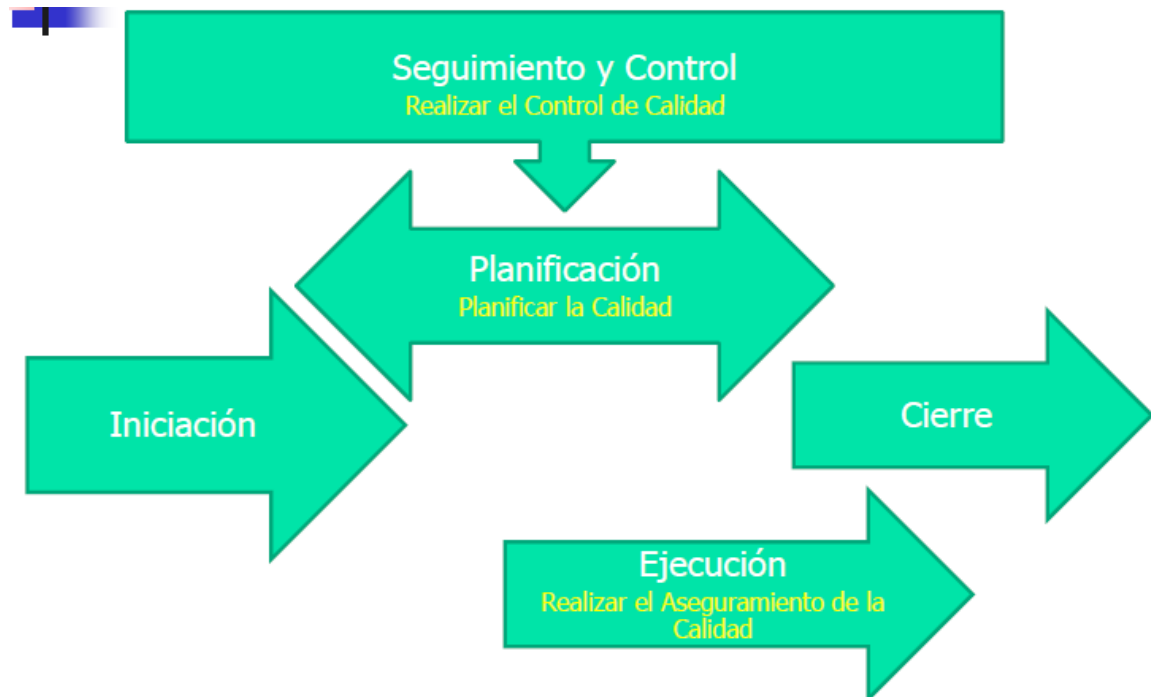
GESTIÓN DE LA CALIDAD

- ◎ Calidad en un proyecto:
 - ◎ Prevenir errores y defectos
 - ◎ Evitar el re-trabajo o re-proceso
 - ◎ Tener un cliente satisfecho
- ◎ Se debe considerar:
 - ◎ Satisfacción del cliente
 - ◎ Prevención antes reactividad
 - ◎ Mejora continua
 - ◎ Responsabilidad de la Dirección



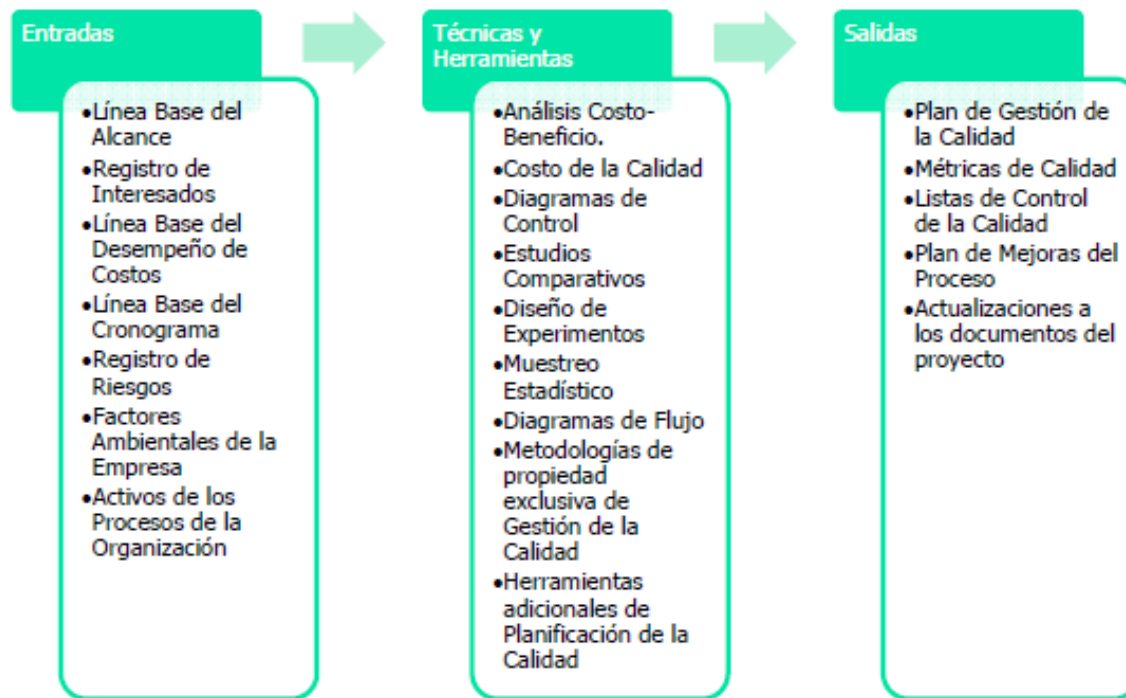
GESTIÓN DE LA CALIDAD

- Procesos necesarios para cumplir las expectativas del cliente, así como el control y seguimiento del proyecto.



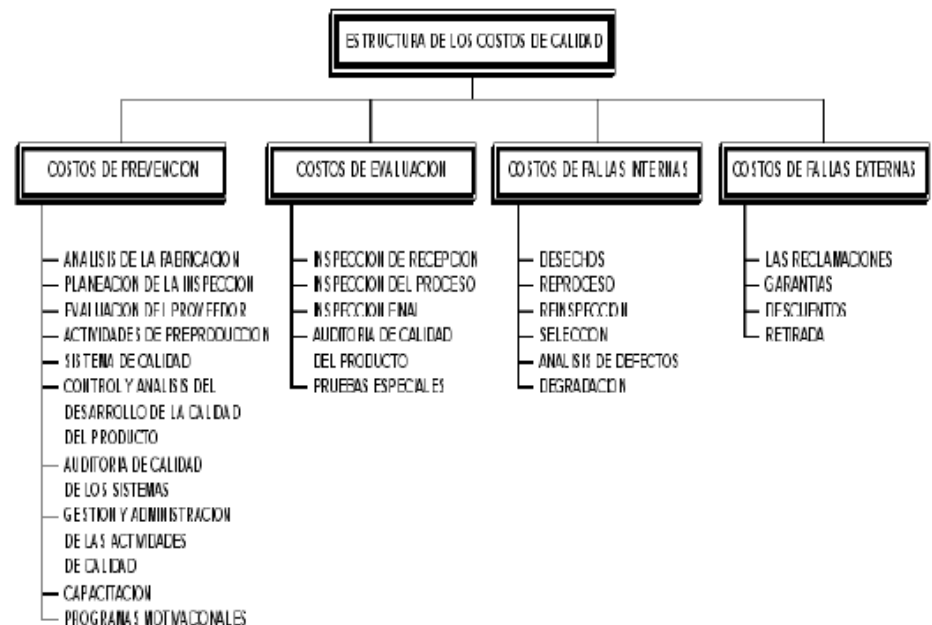
GESTIÓN DE LA CALIDAD

🎯 Planificar la calidad



GESTIÓN DE LA CALIDAD

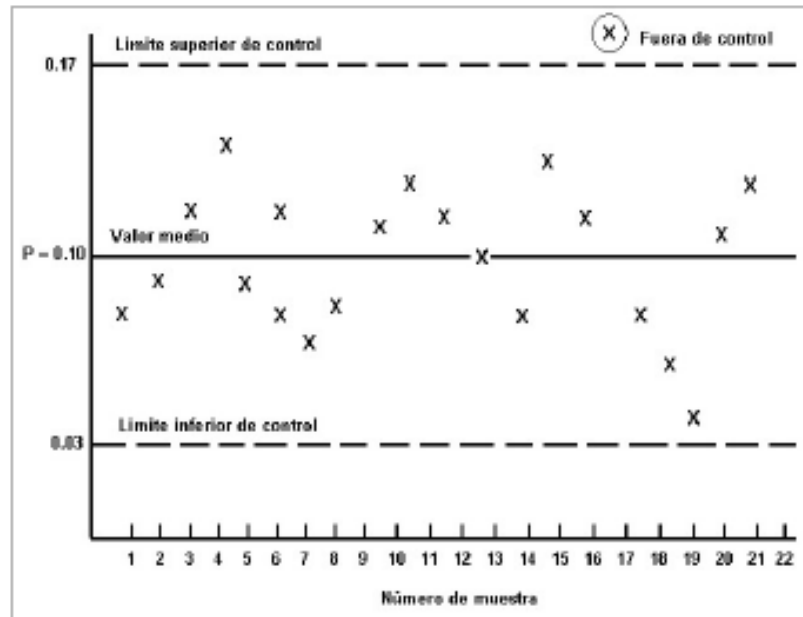
- ◎ Costo de la Calidad (COQ):
 - ◎ Costos incurridos para prevenir el incumplimiento de los requisitos o por no cumplir con los requisitos.



GESTIÓN DE LA CALIDAD

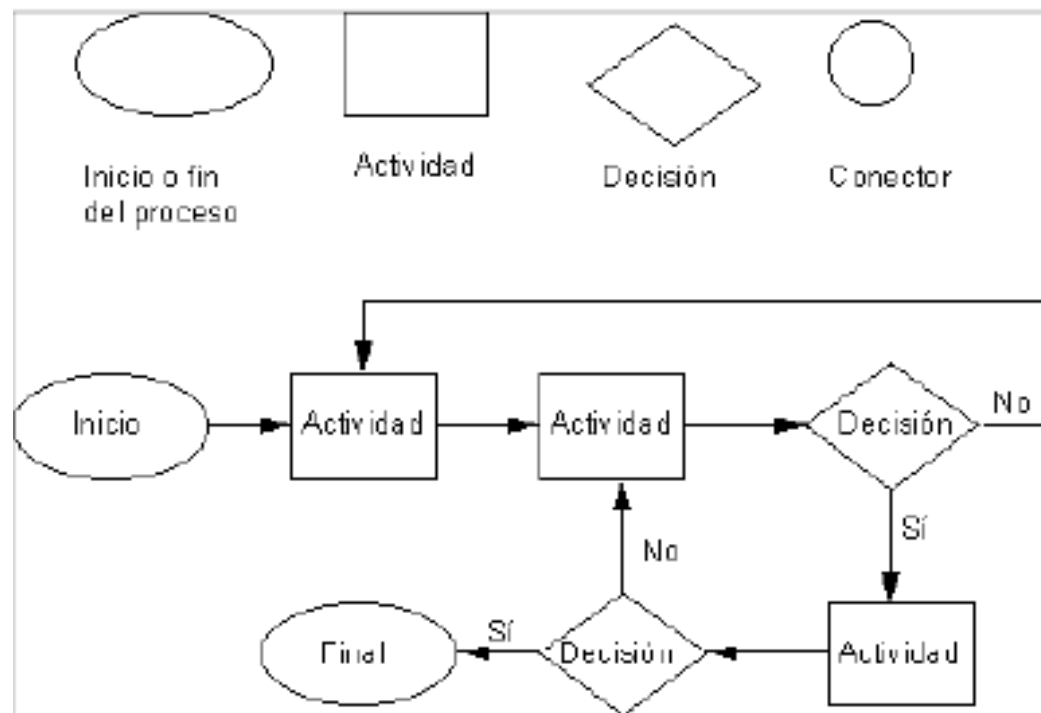
Diagrama de Control:

- Se utilizan para medir la estabilidad del proceso o si tiene un desempeño predecible.



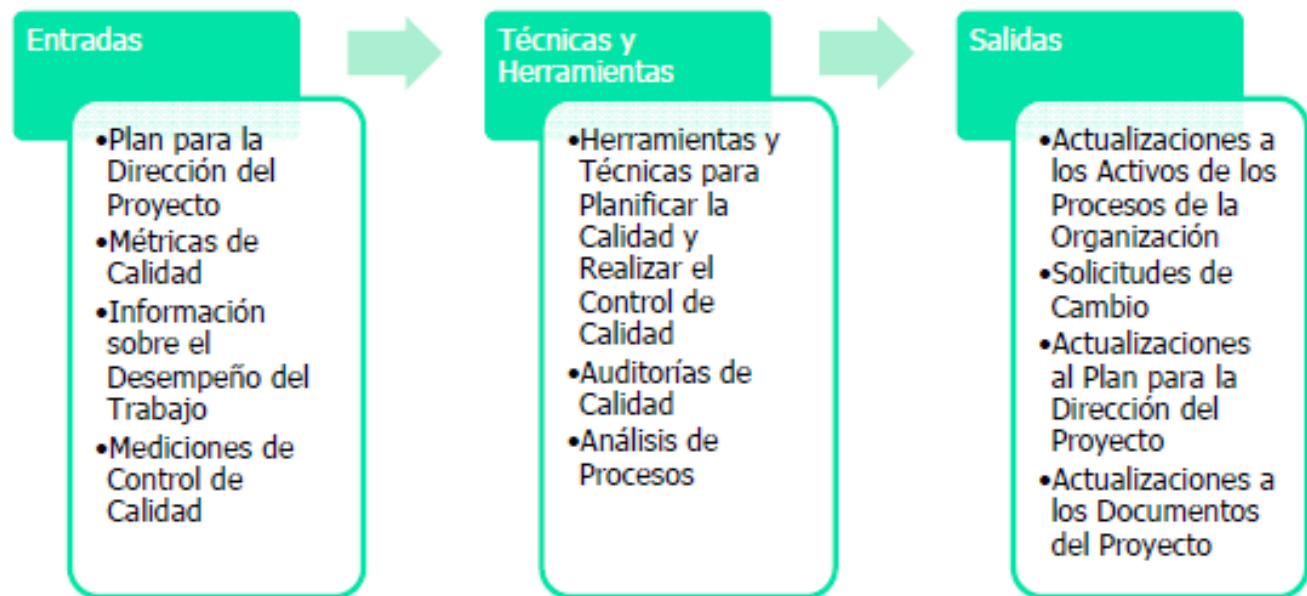
GESTIÓN DE LA CALIDAD

Diagrama de Flujo:



GESTIÓN DE LA CALIDAD

⊙ Aseguramiento de la calidad



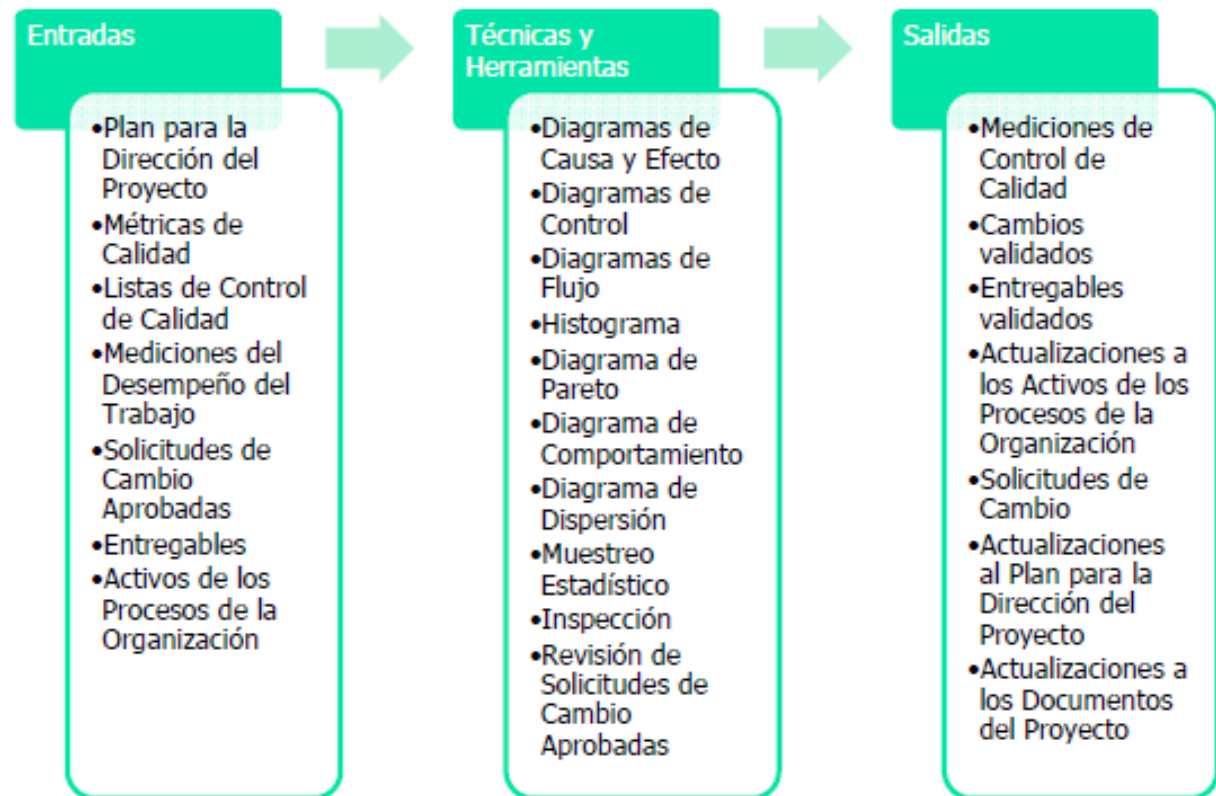
GESTIÓN DE LA CALIDAD

- ◎ Auditoria de Calidad
 - ◎ Evaluar la necesidad de mejoramiento.
 - ◎ Grado de conformidad o no conformidad del Plan de Gestión.
 - ◎ Verificar el cumplimiento del Plan de Gestión de Calidad



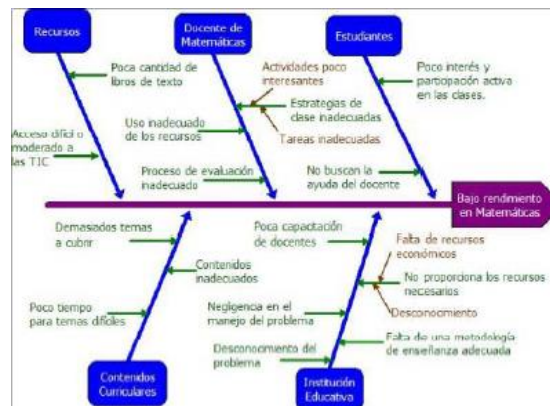
GESTIÓN DE LA CALIDAD

Control de la Calidad

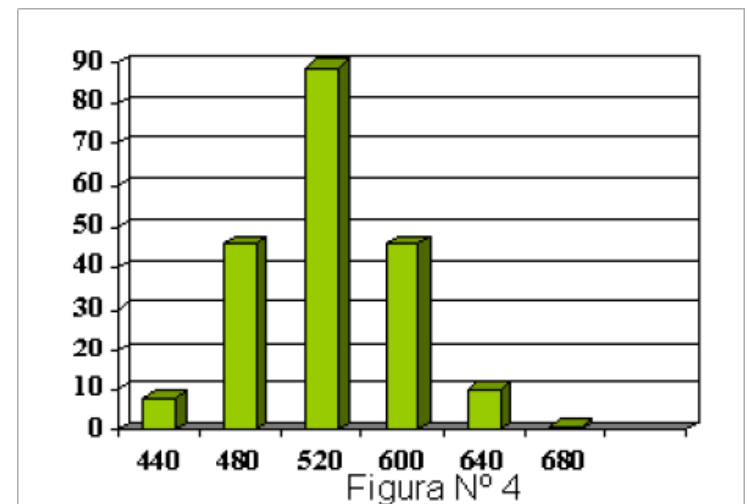


GESTIÓN DE LA CALIDAD

Diagrama de Ishikawa



Histograma



PRUEBAS DE SOFTWARE

Pruebas: (1) El proceso de operar un sistema o componente bajo condiciones especificadas, observando o registrando los resultados y haciendo una evaluación de algún aspecto del sistema o componente.

(2) El proceso de analizar un elemento de software para detectar las diferencias entre las condiciones existentes (pulgas) y requeridas y evaluar las características funcionales de los elementos de software.

IEEE Std 612-1990



PRUEBA DE SOFTWARE

Caso de Prueba: (1) Un conjunto de entradas, condiciones de ejecución y resultados esperados desarrollados para un objetivo particular, tal como el ejercicio de un camino de un programa o para verificar la conformidad con un requerimiento específico.

(2) Documentación que especifica las entradas, resultados esperados y un conjunto de condiciones de ejecución para un elemento de prueba.

IEEE Std 612-1990



PRUEBA DE SOFTWARE

Procedimiento de Prueba: (1) Instrucciones detalladas para la configuración del ambiente de prueba, la ejecución y la evaluación de los resultados de un caso de prueba dado.

(2) Un documento que contiene un conjunto de instrucciones asociadas

(3) Documentación que especifica una secuencia de acciones para la ejecución de una prueba

IEEE Std 612-1990



TIPOS DE PRUEBAS

Pruebas de caja negra – Pruebas funcionales

- (1) Pruebas que ignoran el mecanismo interno de un sistema o componente y se enfocan en solamente en las salidas generadas en respuesta a las entradas seleccionadas y las condiciones de ejecución.
- (2) Pruebas conducidas para evaluar la conformidad de un sistema o componente con los requerimientos funcionales especificados.

IEEE Std 612-1990



TIPOS DE PRUEBAS

Pruebas de caja blanca – Pruebas estructurales

- (1) Pruebas que toman en cuenta el mecanismo interno de un sistema o componente

IEEE Std 612-1990

Pruebas de caja gris

Son una combinación de pruebas de caja negra y pruebas de caja blanca. El ingeniero de pruebas estudia los requerimientos y se comunica con el desarrollador para entender la estructura interna del sistema y seleccionar los casos de prueba más eficientemente. Lewis, W. SW Testing & Continuous Quality Improvement.

En las pruebas de caja gris nos asomamos dentro de la caja solo lo suficiente para entender cómo funciona la solución y seleccionar las pruebas de caja negra de una forma más efectiva. Copeland, L. A practitioner's guide to SW Test Design



TÉCNICAS DE PRUEBAS

Table 1. C: Decomposition for Test Techniques			
C. Test Techniques	C1: (criterion "base on which tests are generated")	C1.1 Based on tester's intuition and experience	Ad hoc
		C1.2 Specification-based	Equivalence partitioning
			Boundary-value analysis
			Decision table
			Finite-state machine-based
			Testing from formal specifications
		Random testing	
		C1.3 Code-based	Reference models for code-based testing (flow graph, call graph)
			Control flow-based criteria
			Data flow-based criteria
		C1.4 Fault-based	Error guessing
			Mutation testing
	C1.5 Usage-based	Operational profile	
		SRET	
	C1.6 Based on nature of application	Object-oriented testing	
		Component-based testing	
		Web-based testing	
		GUI testing	
		Testing of concurrent programs	
		Protocol conformance testing	
		Testing of distributed systems	
		Testing of real-time systems	
		Testing of scientific software	
		C2: (criterion "ignorance or knowledge of implementation")	C2.1 Black-box techniques
	Boundary-value analysis		
	Decision table		
	Finite-state machine-based		
Testing from formal specifications			
Error guessing			
Random testing			
Operational profile			
SRET			
C2.2 White-box techniques	Reference models for code-based testing (flow graph, call graph)		
	Control flow-based criteria		
	Data flow-based criteria		
	Mutation testing		
C3 Selecting and combining techniques			Functional and structural
		Coverage and operational/Situation effect	

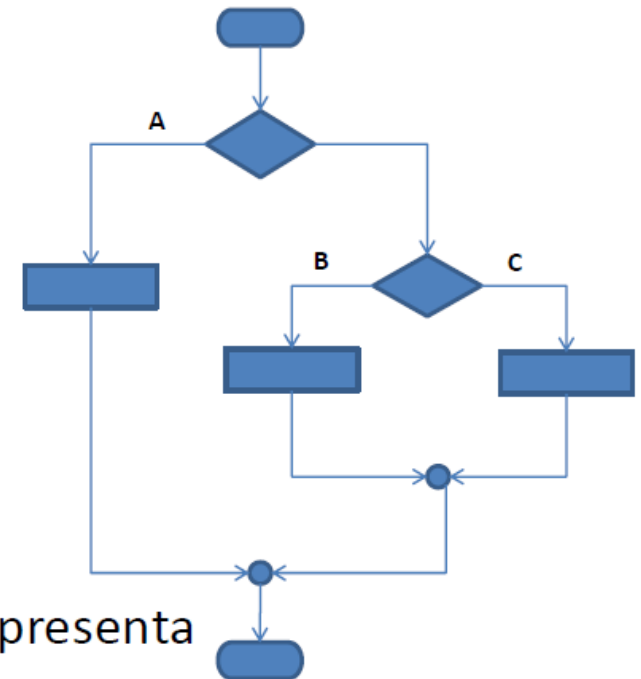


TÉCNICAS DE PRUEBAS

Particiones de Equivalencia

Ejemplo:

$$\text{interés} = \begin{cases} 0, & \text{saldo} < \text{C}500.000 & \text{A} \\ 0.02 \times \text{saldo}, & \text{C}500.000 \leq \text{saldo} < \text{C}1.000.000 & \text{B} \\ 0.025 \times \text{saldo}, & \text{saldo} \geq \text{C}1.000.000 & \text{C} \end{cases}$$



Cada clase de equivalencia representa un camino de ejecución



TÉCNICAS DE PRUEBAS

Análisis de valores de frontera

- Los valores de frontera delimitan las clases de equivalencia.

Ejemplo:

Si *salarioMensual* es mayor que $\text{€}750.000$ y menor que $\text{€}1.250.000$ entonces, $\text{impuesto} = (\text{salarioMensual} - \text{€}750.000) * 0.1$

Gráficamente, se tienen 5 clases de equivalencia:



$\text{€}MaxVal$ representa el valor máximo que puede representar la variable *salarioMensual*



TÉCNICAS DE PRUEBAS

Tablas de decisión

- Muchos requerimientos tienen la forma de condicionales que se pueden representar como decisiones booleanas.

Ejemplo:

Si condiciónA y condiciónB entonces realizar AcciónX

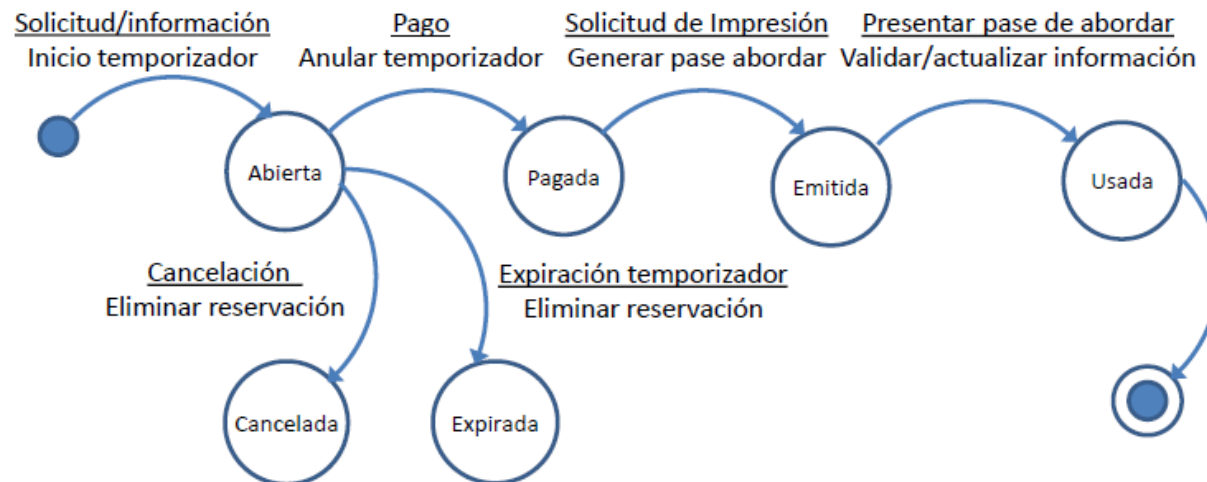
Un requerimiento como este genera la siguiente tabla de verdad:

condiciónA	condiciónB	Resultado
Falso	Falso	No acción
Falso	Verdadero	No acción
Verdadero	Falso	No acción
Verdadero	Verdadero	AcciónX



TÉCNICAS DE PRUEBAS

Pruebas basadas en máquinas de estados finitos
Ejemplo: reservación de pasaje aéreo.



Las transiciones son generadas por eventos y pueden disparar acciones o no.



REVISIÓN

Un proceso o reunión durante la cual un producto de software es presentado a personal del proyecto, administradores, usuarios, clientes, representantes de los usuarios u otras partes interesadas para obtener sus comentarios o aprobación.

IEEE 610.12-1990 (R2002)

Objetos susceptibles de revisión

- Algunos productos de software que son susceptibles de un proceso de revisión son:
 - ✓ Planes de proyecto (desarrollo, verificación, administración de la configuración etc.)
 - ✓ Requerimientos de usuario
 - ✓ Requerimientos de software
 - ✓ Diseño
 - ✓ Código fuente
 - ✓ Casos de prueba
 - ✓ Procedimientos de prueba
 - ✓ Resultados de prueba



TIPOS DE REVISIONES

- **Inspecciones**

Exámenes visuales de los productos de software para detectar e identificar anomalías, incluyendo errores y desviaciones de los estándares y las especificaciones. Las inspecciones siguen un proceso bien definido par encontrar defectos.

- **Caminatas**

Análisis estático en el que un programador o diseñador conduce al equipo de revisión a través de su producto de software a fin de encontrar errores o violaciones a los estándares.

- **Auditorías**

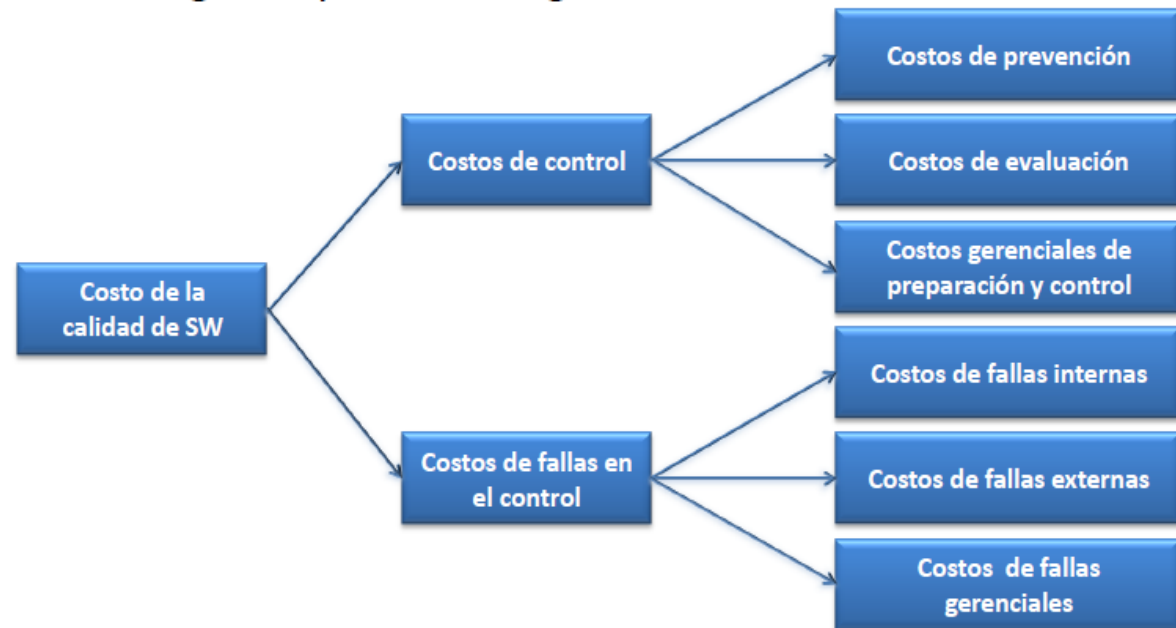
Examen independiente de un producto o proceso de software a fin de determinar si este cumple con las especificaciones, estándares, términos contractuales u otros criterios.



COSTOS DE CALIDAD

Modelo de Galin

- Galin propone un modelo de costos de la calidad que consiste en dos categorías y 6 sub-categorías



COSTOS DE CALIDAD

Modelo de Dobbins

Dobbins (en Schulmeyer & McManus: Handbook of SW Quality Assurance) propone llevar a cabo el diseño del sistema de costos de la calidad haciendo un análisis de cada actividad identificando:

1. Entradas
2. Valor agregado de la actividad
3. Salidas
4. Requerimientos del cliente para cada actividad
5. Requerimientos para los proveedores.
6. Métricas
7. Nivel de esfuerzo (horas-personal)
8. Categoría de costos



MÉTRICAS DE LA CALIDAD

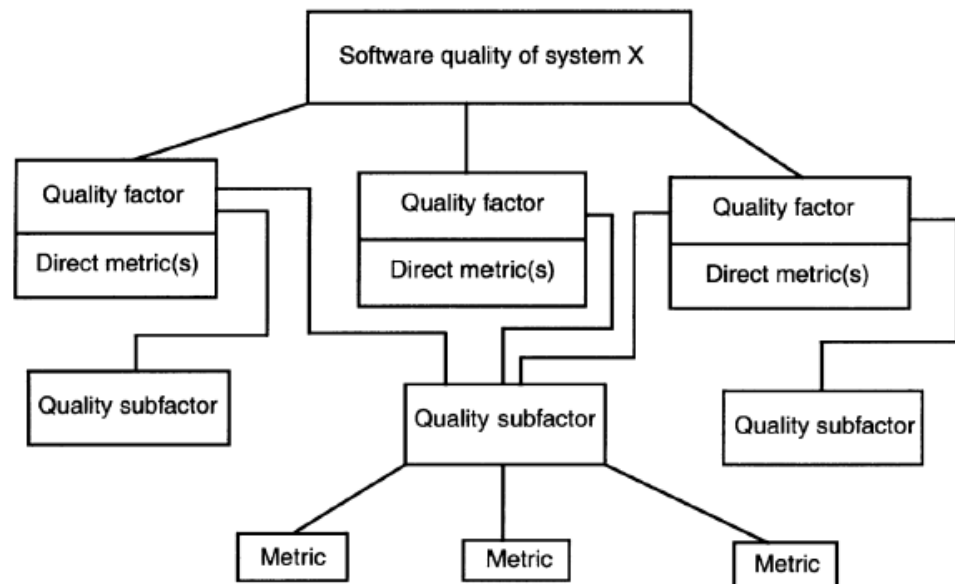
Métrica de calidad

- 1) Medida cuantitativa del grado en que un elemento posee un atributo de calidad dado.
- 2) Una función cuyas entradas son datos de software y cuya salida es un valor numérico único que puede ser interpretado como el grado en que el proceso de software posee un atributo de calidad dado.



MÉTRICAS DE CALIDAD

El estándar IEEE 1061-1998



El marco de trabajo de métricas de calidad

IEEE Std. 1061-1998



MÉTRICAS DE CALIDAD

Categorías y sub-categorías

Existen dos grandes categorías de métricas de calidad:

- Métricas de proceso
Describen la calidad del proceso de desarrollo.
- Métricas de producto
Describen las características del producto una vez que este se encuentra en servicio.
- No existe una definición estandarizada para las sub-categorías. Diferentes autores subdividen estas dos categorías en diferentes sub-categorías.



MÉTRICAS DE CALIDAD

Clasificación de Kan

Métricas intrínsecas de producto – MTTF y MTBF

- El MTTF se usa en sistemas que no pueden ser reparados.
- El tiempo medio entre fallas (MTBF por sus siglas en inglés) se usa para artículos que pueden ser reparados.

$MTBF = \text{Tiempo total observado} / \text{número de fallas}$

$MTBF = MTTF + MTTR$, donde MTTR: Tiempo medio para reparación

- Estas métricas son ampliamente utilizadas en sistemas de manufactura.
- El MTTF y el MTBF no son métricas de confiabilidad.
- No existen modelos de confiabilidad estándar para software.



MÉTRICAS DE CALIDAD

Clasificación de Kan

Métricas intrínsecas de producto – Densidad de Defectos

- Las métricas de densidad de defectos se refieren al número de defectos por unidad de tamaño de software.
- Las métricas de tamaño de software más usadas son el número de líneas de código y los puntos de función.
- A fin de poder compararlas apropiadamente, estas métricas deben definir el período de observación. Usualmente se usan períodos de 1-4 años.
- Finalmente se debe definir qué es un defecto.



MÉTRICAS DE CALIDAD

Clasificación de Galin

- Servicios de soporte (*Help Desk*)

Métrica	Forma de cálculo
Densidad de llamadas	Número de llamadas en un año de servicio/total de líneas de código
Densidad de llamadas ponderada	Número de llamadas ponderadas en un año de servicio/total de líneas de código
Densidad de llamadas ponderada	Número de llamadas ponderadas en un año de servicio/punto de función

La ponderación se lleva a cabo con base en la severidad de la llamada.



MÉTRICAS DE CALIDAD

- Servicios de soporte

Métrica	Forma de cálculo
Densidad de llamadas	Número de llamadas en un año de servicio/total de líneas de código
Densidad de llamadas ponderada	Número de llamadas ponderadas en un año de servicio/total de líneas de código o punto de función
Éxito de servicios de soporte	Número de reportes resueltos en un año de servicio
Severidad promedio de los reporte	Número de llamadas ponderadas en un año de servicio/Número de llamadas
Efectividad del centro de servicio	Horas de servicio del centro de soporte/número de llamadas en un año



MÉTRICAS DE CALIDAD

Clasificación de Galin

- Métricas de mantenimiento correctivo

Métrica	Forma de cálculo
Densidad de fallas del sistema	$\text{Número de fallas en un año} / \text{Número de líneas de código o puntos de función}$
Densidad ponderada de fallas del sistema	$\text{Número de fallas ponderadas en un año} / \text{Número de líneas de código o puntos de función}$
Disponibilidad	$(\text{Horas en servicio} - \text{horas fuera de servicio}) / \text{Horas de servicio}$
Indisponibilidad	$\text{Horas fuera de servicio} / \text{Horas de servicio}$
Productividad de mantenimiento	$\text{Horas de mantenimiento} / \text{Número de líneas de código o puntos de función}$
Efectividad de mantenimiento	$\text{Horas de mantenimiento} / \text{Número de fallas en un año}$



MÉTRICAS DE PROCESOS

Modelo de Galin

- Galin clasifica las métricas de proceso en tres categorías:
 - ✓ Métricas de calidad del proceso de software
 - ✓ Métricas de tiempo del proceso de software
 - ✓ Métricas de productividad del proceso de software



EJERCICIO 5.6

- ◎ Para el proyecto que definió en la actividad 2.1,
 - ◎ Establecer el flujo de control sobre el proyecto.
 - ◎ Determinar las técnicas que se utilizarán para gestionar la calidad
 - ◎ Establecer una lista de chequeo para el control de calidad.
 - ◎ Establecer un plan de auditorías.
 - ◎ Utilizar los formatos brindados
- ◎ Tiempo 20 minutos.



MUCHAS GRACIAS

